

L19: SimPy Store, FilterStore, and Container

Simulation Without a Black Box

Outline

Learning Objectives

Resource vs. Store vs. Container

simpy.Store

simpy.FilterStore

simpy.Container

Batching Pattern

Inventory Model with Container

Key Takeaways

Learning Objectives

By the end of this lecture you will be able to:

- Distinguish Resource, Store, and Container and choose the right one.
- Use `simpy.Store` for FIFO queues of heterogeneous objects.
- Use `simpy.FilterStore` to retrieve items by predicate.
- Use `simpy.Container` for continuous bulk quantities.
- Model batching (accumulate until N items) and an (s,S) inventory system.

Resource vs. Store vs. Container: When to Use Which

Type	Contents	Typical use
Resource	Anonymous capacity slots	Servers, machines, nurses
Store	Heterogeneous Python objects (FIFO)	Job queue, message buffer, patient records
FilterStore	Objects, retrieved by predicate	Warehouse with SKUs, organ donor matching
Container	Scalar level (float)	Fuel tank, inventory on-hand, battery charge

Key distinction

Resource tracks *who is using it*; Store holds *items* (put/get); Container holds a *level* (put/get with amounts).

simpy.Store: Heterogeneous FIFO Queue

```
import simpy, numpy as np
from dataclasses import dataclass

@dataclass
class Job:
    job_id: int
    size: float  # processing time

env = simpy.Environment()
queue = simpy.Store(env)  # unbounded FIFO

def producer(env, queue, rng):
    jid = 0
    while True:
        yield env.timeout(rng.exponential(1.0))
        job = Job(job_id=jid, size=rng.exponential(0.8))
        yield queue.put(job)  # put into store
        jid += 1

def worker(env, queue, done):
    while True:
        job = yield queue.get()  # wait until item available
        yield env.timeout(job.size)
        done.append(job.job_id)
```

simpy.FilterStore: Retrieve by Predicate

```
import simpy
```

```
env = simpy.Environment()
wh = simpy.FilterStore(env)    # warehouse
```

```
# Restock: put items with SKU labels
```

```
wh.put({"sku": "A", "qty": 10})
```

```
wh.put({"sku": "B", "qty": 5})
```

```
wh.put({"sku": "A", "qty": 7})
```

```
def order_processor(env, wh, target_sku):
    # Blocks until an item with matching SKU is available
    item = yield wh.get(
        lambda x: x["sku"] == target_sku)
    print(f"t={env.now}: fulfilled order for {target_sku}")
```

- The predicate is evaluated against each item in order; blocks until one matches.
- Use for *discriminating consumers*: organ matching, specialised parts, blood type.
- Performance degrades if the store is large and matches are rare — consider indexing.

simpy.Container: Bulk Resources

```
import simpy, numpy as np

env = simpy.Environment()
tank = simpy.Container(env, capacity=1000, init=500)

def consumer(env, tank, rng):
    """Continuously consume fuel at a random rate."""
    while True:
        amount = rng.uniform(10, 50)
        yield tank.get(amount)           # blocks if level < amount
        yield env.timeout(1.0)          # consume over 1 time unit

def refueller(env, tank, refill_amount=400):
    """Refuel when tank drops below 200."""
    while True:
        yield env.timeout(5.0)
        if tank.level < 200:
            yield tank.put(
                min(refill_amount, tank.capacity - tank.level))
```

`tank.level` gives the current scalar quantity; `get` and `put` block if level is insufficient or capacity is exceeded.

Batching: Accumulate Until N Items Arrive

```
BATCH_SIZE = 5
```

```
def batcher(env, in_store, out_store):  
    """Collect BATCH_SIZE items then release as one batch."""  
    while True:  
        batch = []  
        for _ in range(BATCH_SIZE):  
            item = yield in_store.get()  
            batch.append(item)  
            # All BATCH_SIZE collected --- send downstream  
        yield out_store.put(batch)  
        print(f"t={env.now:.1f}: batch of {BATCH_SIZE} dispatched")
```

- Batching is common in manufacturing (ovens, autoclaves) and transportation (buses, trucks).
- The batcher blocks until the full batch arrives — creates a dependency on upstream rate.
- Timeout-based batching: release after N items or T time, whichever comes first, using `yield store.get() | env.timeout(T)`.

(s,S) Inventory: Container with Depletion and Replenishment

```
s, S = 20, 100    # reorder point, order-up-to level
L     = 5.0       # replenishment lead time

def demand(env, inv, rng, stats):
    while True:
        yield env.timeout(rng.exponential(1.0))
        d = int(rng.integers(1, 6))          # demand 1..5 units
        if inv.level >= d:
            yield inv.get(d)
            stats["filled"] += d
        else:
            stats["short"] += d - inv.level   # backlog
            yield inv.get(inv.level)

def reviewer(env, inv):
    while True:
        yield env.timeout(1.0)                # daily review
        if inv.level <= s:
            yield env.timeout(L)              # lead time
            yield inv.put(S - inv.level)      # order up to S
```

Key Takeaways

Core ideas from L19

1. **Resource**: capacity slots (servers). **Store**: heterogeneous objects (jobs). **Container**: scalar level (inventory, fuel).
2. `Store.get()` and `Container.get()` *block* until an item or sufficient level is available — use this for demand that waits on supply.
3. **FilterStore** allows selective retrieval by predicate; use sparingly on large stores.
4. **Batching**: loop N `get()` calls inside a generator to accumulate items.
5. The (s,S) inventory pattern: `Container` depletion from demand, put after lead-time timeout.

Next: L20 — Multi-stage service networks, Burke's theorem, and Jackson networks.